

Route B: An Asset-Informed Geometric Baseline for Language-Conditioned Manipulation

Anonymous Authors

Abstract—Language-conditioned manipulation requires semantic grounding, geometric reasoning, and reliable transitions between physical interactions. We study how far an explicit, asset-informed system can address these requirements without training an action policy on robot demonstrations. Route B combines a constrained language compiler, frozen visual grounding, calibrated RGB-D reconstruction, and a sequential dispatcher for grasp-place and contact skills. Its geometric state distinguishes observed surfaces from offline object priors and maintains the displacement between the gripper and the carried object during placement. Contact skills use robot measurements and visual checks to manage articulation and recovery. The policy interface excludes task-success feedback, while an external evaluator records whether the official predicate is ever satisfied. On 400 fixed initializations spanning 40 LIBERO tasks, the deployed system succeeds in 347 trials (86.8%). Controller completion and external success disagree in 46 trials, demonstrating why these outcomes must be reported separately. The study characterizes a reproducible geometric baseline with explicit asset and calibration assumptions; it does not establish generalization to unseen assets or superiority over learned policies.

I. INTRODUCTION

Natural-language instructions provide a compact description of a manipulation goal, but leave much of the physical procedure unspecified. To place a bowl inside a drawer, a robot must identify the intended object, infer accessible geometry, establish a stable grasp, maintain clearance during transport, and release the object in a suitable region. Articulation introduces an additional difficulty: the robot must distinguish productive motion from mechanical resistance and decide when to release or reacquire contact. A plausible semantic interpretation alone does not resolve these questions.

Large vision-language-action policies offer one route to this capability. OpenVLA [1] adapts pretrained representations to action prediction, while SmolVLA [2] studies a more compact architecture and efficient execution. These approaches motivate an accompanying question: what performance can an explicit system achieve when visual grounding is pretrained but the action policy is constructed from geometry and hand-engineered skills? Answering this question requires disclosing the information available to the system, including asset knowledge and calibration, rather than equating the absence of policy training with the absence of prior information.

We present Route B (Fig. 1), an asset-informed baseline for language-conditioned tabletop manipulation. The system compiles supported instructions into ordered skill programs and binds their entities to geometry reconstructed from two

calibrated RGB-D views. It uses a frozen detector together with a gallery derived from public simulator assets. A dispatcher preserves state across grasp-place segments and invokes specialized contact controllers for drawers, doors, knobs, and pushing. Robot actions are bounded Cartesian pose increments with a gripper command.

The central engineering issue is continuity across physical stages. A semantically correct target can still be unreachable with the current wrist orientation; a successful close command need not establish retention; and a gripper placed at a receptacle center need not center the object being carried. Route B therefore maintains explicit object-to-gripper displacement, re-observes relevant geometry at selected transitions, and uses local completion checks before advancing its program. These checks are fallible estimates, separate from benchmark success.

We evaluate the deployed implementation on the Spatial, Object, Goal, and Long suites of LIBERO [3]. Each suite contains ten evaluated tasks, with ten official initial states per task. The evaluation uses the existing controller, fixed assets, and a single seed; its scope is a system study on known benchmark assets and instruction forms. We make three contributions:

- 1) An integrated language-to-geometry execution architecture that declares its offline priors and separates policy observations from external scoring.
- 2) A documented account of how carried-object geometry, visual checks, and contact-specific controllers are composed in the deployed system.
- 3) A record-based evaluation reporting task-level outcomes, execution costs, and disagreement between internal completion and external success.

The contribution is the integrated baseline and its empirical characterization. The present evidence does not isolate the causal benefit of each component. We retain this distinction throughout the paper so that the system can support subsequent controlled comparisons.

II. RELATED WORK

A. Language, Programs, and Geometric Planning

Task and motion planning combines discrete decisions with continuous geometric feasibility [4]. Language-conditioned systems extend this interface: SayCan connects language choices to available skills and their affordances [5], while Code as Policies generates executable programs that call perception and control interfaces [6]. Route B uses a deterministic grammar and a fixed skill vocabulary. This

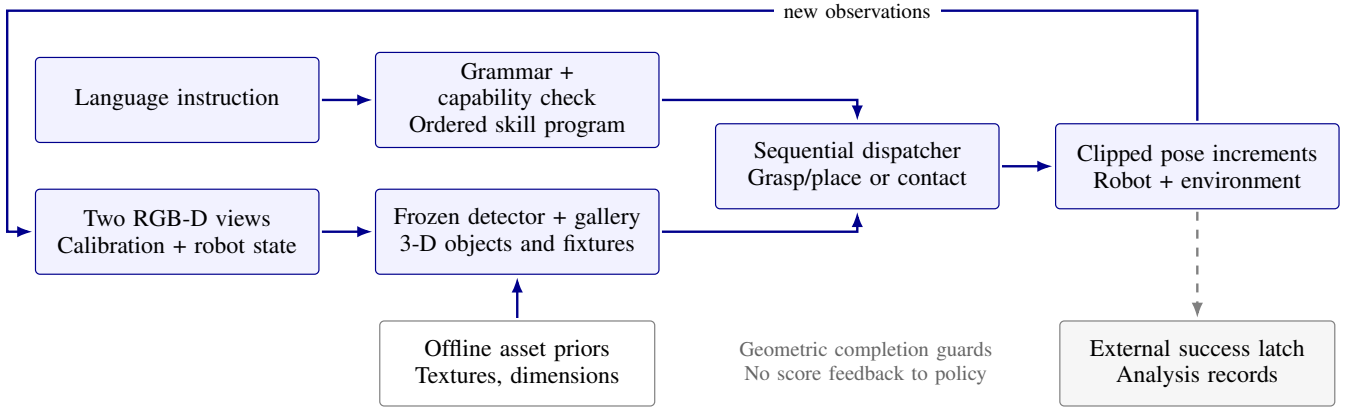


Fig. 1. System organization. Language selects the ordered skill program; calibrated observations and offline asset priors support geometric grounding. The dispatcher invokes grasp-place or contact control and receives subsequent observations. The external evaluator records task success without returning it to the policy. The diagram summarizes software data flow, not a verified process-security boundary.

makes the represented program inspectable, but restricts language coverage to implemented constructions and validated entities.

VoxPoser constructs spatial value maps to guide motion [7]. ReKep expresses manipulation objectives through relational keypoint constraints [8]. These works show the utility of intermediate geometric representations. Our implementation uses object and fixture geometry, carried-object offsets, and specialized finite-state controllers. It does not synthesize arbitrary constraint programs or perform a unified optimization over the complete task. Its behavior depends on the coverage and calibration of its manually implemented skills.

B. Pretrained Perception and Action Policies

Grounding DINO supports text-conditioned object detection [9]. Route B uses a frozen local checkpoint as one source of region proposals; the downstream geometric and gallery components remain essential to the implemented pipeline. In particular, public object textures and collision dimensions provide priors that are stronger than an image-only input setting.

OpenVLA [1] and SmolVLA [2] learn action generation from robot data. Route B provides a complementary, explicitly engineered control baseline. The distinction concerns how actions are obtained; it does not imply an absence of learning in the pretrained detector or an absence of development on benchmark scenarios. Our evaluation is not a matched comparison to either policy, because sensing, asset access, seeds, and stopping conventions differ.

C. Benchmark Interpretation

LIBERO was introduced to study knowledge transfer in lifelong robot learning [3]. We use its four commonly evaluated ten-task suites to characterize a fixed manipulation system, rather than measuring continual learning or forgetting. The choice of ten trials per task follows the sample count used in SmolVLA’s simulation study [2]. Matching that count alone does not reproduce its training or evaluation conditions. We specify the remaining conditions in Section IV.

III. SYSTEM

A. Observation and Prior Information

At step t , the policy receives

$$o_t = (\{I_t^c, D_t^c, K^c, T_{WC,t}^c\}_{c \in \{a,w\}}, r_t), \quad (1)$$

where a and w denote the fixed and wrist cameras, I is RGB, D is metric depth, K is calibration, and T_{WC} maps camera coordinates into the world frame. Robot measurements r_t include end-effector pose, joint state, gripper state, and available force/torque measurements. The grasp-place adapter uses pose, joint positions, and gripper width; contact execution can access the broader robot measurement interface.

These are simulated, calibrated measurements. In particular, camera extrinsics and proprioception are supplied by the simulator adapter, and depth is not an estimate from monocular RGB. The policy observation types contain no object pose labels, segmentation IDs, BDDL predicates, reward, or task-success flag. Task identifiers and initial-state indices remain with the evaluator; reset passes the instruction and a nonsemantic episode token. This is a software interface property supported by code inspection, not a claim of adversarial isolation.

Offline information is represented by a prior collection $\mathcal{P}$. It includes object texture histograms, dimensions derived from public collision geometry, robot geometry, class-specific handling rules, and motion thresholds. The evaluated controller is therefore $a_t = \pi(o_{\leq t}, \ell; \mathcal{P})$, where ℓ is the instruction. No action-policy fitting or demonstration loading occurs in this evaluation. The detector’s pretraining and the controller’s manual development remain part of the system’s knowledge.

B. Constrained Language Compilation

The compiler maps supported language to

$$\mathcal{G}(\ell) = (s_1, \dots, s_M), \quad s_j = (k_j, e_j^{\text{src}}, e_j^{\text{dst}}, \rho_j). \quad (2)$$

Here k_j is a skill, the entity bindings specify source and destination, and ρ_j represents a placement relation or instance

selector. The vocabulary contains pick, place-on, place-in, relative placement, stack, open, close, push, turn-on, and turn-off. For example, a two-object basket instruction becomes two successive pick/place pairs. A drawer instruction can produce an articulation segment followed by a manipulation segment.

Bindings retain relations such as *next to*, *between*, or an ordering among visible instances. They do not encode the simulator’s object identity. The complete program undergoes capability checks before motion, including valid skill arguments and consistency of held-object state. Compilation success only establishes representational support; it does not establish physical executability. The deployed grammar is tailored to known LIBERO phrases and an allow-listed entity vocabulary. An optional semantic-resolver interface exists, but no language-model resolver is configured in this run.

C. Visual Grounding and Geometric State

Each calibrated pixel $\tilde{u} = (u, v, 1)^\top$ with valid depth contributes

$$x_W = R_{WC}(D(u, v)K^{-1}\tilde{u}) + p_{WC}. \quad (3)$$

Workspace filtering, support estimation, and connected components yield candidate point sets. The pipeline fits observed bounds and uses texture/size priors to classify and complete partial observations. The gallery computes a color similarity

$$b(h, h_k) = \sum_i \sqrt{h_i h_{k,i}} \quad (4)$$

between normalized HSV histograms. Its score combines color and shape terms, with configured weights 0.4 and 0.6, a softmax temperature of 0.13, and additional class-dependent footprint gates. Grounding DINO supplies language-conditioned boxes that are converted to 3-D hypotheses using depth. Thus a detector box is a proposal, rather than an oracle object mask.

Cross-camera hypotheses with matching labels are clustered in 3-D. For a cluster, the implemented fusion weights are proportional to $\max(q_i, 0.05)\sqrt{n_i}$, where q_i is confidence and n_i is point count. The default center-distance threshold is 4.5 cm. These thresholds are empirical operating choices; nearby repeated objects can still produce ambiguous associations.

The state retains both observed support/bounds and completed geometry. This distinction matters when an object is partially visible: a size prior can stabilize a center estimate, but should not be mistaken for measured free space. Geometric selectors resolve the intended instance from the available candidates and reference objects. At selected phase transitions, new views are requested and checked against prior observations to reduce hand-occlusion-induced jumps. Specialized fixture and cavity routines refine support surfaces, opening regions, and admissible approach geometry.

D. Grasping, Transport, and Placement

Consecutive manipulation skills share a grasp–place controller. Its phases include acquisition, pregrasp motion, approach, finger closure, retention checking, lift, transport,

descent, release, and retreat. Transitions combine pose tolerances, measured gripper width, visual evidence, and bounded retry logic. Different grasp modes account for exterior pinch, rim pinch, and interior expansion; mode choice depends on the object and observed accessibility. These are mechanical templates with specialized branches.

For target pose (p^*, R^*) , the basic servo emits

$$a_t = \text{clip}_{[-1,1]} \begin{bmatrix} (p^* - p_t)/s_p \\ \text{Log}(R^* R_t^\top)^\vee / s_R \\ g_t \end{bmatrix}. \quad (5)$$

The general grasp–place defaults are $s_p = 0.05$ m and $s_R = 0.25$ rad; cavity handling can select another rotation scale. The nominal pose tolerances are 8 mm and 0.12 rad. These are normalization scales and transition tolerances, not measured tracking-error bounds or a continuous-time stability guarantee.

The gripper and carried object generally have different centers. Let $\hat{d}_t$ be the estimated world-frame vector from the end effector to the carried-object reference point. For desired object location p_o^* , placement planning uses the geometric relation

$$p_{ee}^* = p_o^* - \hat{d}^*, \quad \hat{d}^* = \Delta R \hat{d}_t \quad (6)$$

when the planned wrist motion rotates the held geometry by ΔR . The implementation adds support, release-height, aperture, and fixture-specific corrections to this relation. Offset estimates may come from precontact geometry or refreshed RGB-D evidence; their origin is retained in the execution trace. This allows inspection of placement decisions without assuming that all offset estimates are equally reliable.

Release is followed by sensor-based verification. A local geometric check can reject a placement even when the benchmark predicate has already been true, or accept a relation that the benchmark does not recognize. Section V-B quantifies this distinction rather than using controller completion as the reported success metric.

E. Contact Skills and Sequential Execution

The dispatcher separates consecutive grasp–place steps from atomic contact skills. Drawer and door interactions use observed fixture geometry to construct approach, engagement, motion, release, and retreat stages. Knob and push skills use their own target detectors and motion templates. Shared episode state includes an initial robot-pose reference and, where applicable, an observed drawer anchor for subsequent placement.

Contact evidence combines position error, lack of measured progress, and changes in end-effector force. One implemented guard requires a minimum contact duration and bounded position error, together with either a stalled error reduction or sufficient force change. Such a guard indicates a candidate contact condition; it is not the task predicate. Specialized post-contact visual checks assess the resulting fixture configuration.

Local recovery is bounded by phase counters and geometric checks. Examples include releasing a rejected grasp,

TABLE I
DECLARED CONDITIONS FOR THE EVALUATED SYSTEM.

Property	Setting
Policy observations	Fixed and wrist RGB-D, camera calibration, robot measurements
Offline priors	Public asset textures/dimensions, robot geometry, hand-written skill rules
Action generation	Geometric state machines and clipped Cartesian increments
Learning during run	None; frozen Grounding DINO checkpoint
Scene truth in policy	No object pose labels, segmentation IDs, or BDDL predicates in observation types
Success feedback	External latch; no score-driven policy stopping
Coverage	Known benchmark assets and implemented instruction grammar

reacquiring a visible handle, or performing a short clearance motion after stalled drawer interaction. These routines preserve the distinction between a robot-side mechanical signal and an estimate of task completion. They also introduce many hand-selected parameters. The present evaluation measures the complete configured controller; it does not establish which recovery branches are necessary or sufficient.

IV. EXPERIMENTAL PROTOCOL

A. Tasks, Initial States, and Runtime

The planned evaluation contains $4 \times 10 \times 10 = 400$ episodes: Spatial, Object, Goal, and Long, ten tasks each, and official initial-state indices 0–9. Long is named `libero_10` in the implementation. Global random generators and the environment configuration use seed 7. The environment adapter preserves or replays the task reset stream when starting at a later initial-state index, because resetting can affect fixture configuration beyond the saved joint state.

We use two 256×256 RGB-D cameras and a Panda with operational-space pose control. A ten-step settling period precedes scored actions. The Spatial/Object/Goal/Long action budgets are 220/280/300/520, matching the numerical budgets in the OpenVLA evaluation script [10]. Our environment seed and success-handling logic differ from that script; its environment utility uses seed 0. The run uses Ubuntu 22.04, Python 3.12, PyTorch 2.11.0 with CUDA 12.8, MuJoCo 3.3.2 [11], robosuite 1.4.0 [12], and hf-libero 0.1.4 on one RTX 5090.

Task processes execute independently under a dispatcher, initially with four workers and then eight. Each worker evaluates ten initializations of one task. Ten Spatial task-00 episodes are reused from a stopped earlier run: they match the fixed index list and the evaluation-affecting configuration, and their source records were frozen before the new campaign. Their inclusion was determined by identity and compatibility, not outcome. The other 390 episodes are newly executed. We report this reuse explicitly and keep timing analyses separate.

B. External Scoring and Internal Completion

Let q_t be the official success predicate observed by the evaluator. With policy-controlled stopping time τ capped at

TABLE II
OBSERVED SUCCESS AND EXECUTION COST. SR USES THE EXTERNAL EVER-SUCCESS PREDICATE. [†]ELAPSED-TIME MEDIAN USES NEWLY EXECUTED EPISODES ONLY; STEPS INCLUDE REUSED RECORDS. TIMES INCLUDE THE RECORDED EPISODE EXECUTION OVERHEAD UNDER CONCURRENT EVALUATION.

Suite	Success	SR (%)	Mean steps	Median $s^{\dagger}$
Spatial	91/100	91.0	147.4	68.7
Object	96/100	96.0	173.0	54.7
Goal	88/100	88.0	180.9	67.2
Long	72/100	72.0	293.3	90.4
Overall	347/400	86.8	198.6	66.5

budget H , the recorded outcome is

$$Y = \bigvee_{t=1}^{\tau} q_t \vee q_{\text{final}}, \quad \tau \leq H. \quad (7)$$

The final query also handles a zero-action stop. Reward and success do not enter the policy observation. The evaluator continues after a success signal until the controller requests stopping or the action budget expires. The score therefore represents *ever reaching* the predicate during this rollout, not necessarily satisfying it at the final frame.

Each record additionally stores terminal controller state, step count, elapsed time, failure message, and geometric traces. The reported success rate is $\hat{p} = N^{-1} \sum_i Y_i$. Equal task sample counts make the overall episode average equal to the macro-average over tasks in the completed run. We retain all planned failures; no outcome-based episode retry or selection is used. Controller-internal retries remain part of a single episode.

C. Scope of the Evidence

The completed campaign passes its record-coverage validation: 400 unique episodes, fixed configuration, existing recorded videos, and matching deployed source checksums. The manuscript’s tables are generated from a frozen projection of the original records, with hashes retaining the connection to their source lines. An older 2,000-episode score exists only as an imported summary in the available package; it is excluded from this paper’s measured results.

All reported rates are descriptive for the chosen initial states. A single seed and ten fixed states per task do not quantify variability across training runs, environment seeds, or new task distributions. We therefore report counts and task-level outcomes instead of presenting a nominal independent-trial confidence interval as evidence of broad generalization.

V. RESULTS AND ANALYSIS

A. Aggregate and Task-Level Performance

The system succeeds in 347 of 400 episodes, for an overall success rate of 86.8% (Table II). Suite success rates are 91.0% for Spatial, 96.0% for Object, 88.0% for Goal, and 72.0% for Long. The newly executed subset succeeds in 337 of 390 trials (86.4%), allowing the effect of including the ten reused records to be inspected directly.

TABLE III

TERMINAL CONTROLLER STATE AND INDEPENDENTLY RECORDED EXTERNAL EVER-SUCCESS. A CONTROLLER FAILURE CAN FOLLOW AN EARLIER BENCHMARK SUCCESS.

Controller terminal state	External success	External failure
Succeeded	312	11
Failed	32	35
Timeout	3	7

Figure 2 shows the task-level distribution. 21 tasks have ten successful recorded trials. The hardest observed task is Long 09, placing a mug inside a microwave and closing it, with 0/10 successes. Its records contain four placement-target localization failures, three placement-verification rejections, and three phase timeouts. Long 08, placing both moka pots on the stove, succeeds in 4/10 trials. These cases expose limitations in multi-stage execution that are hidden by the overall average.

Drawer-related tasks also remain difficult: Goal task 03, opening the top drawer and placing a bowl inside, succeeds in 3/10 trials; Spatial task 04, retrieving a bowl from a top drawer, succeeds in 5/10; and Long task 03, placing a bowl in a bottom drawer and closing it, succeeds in 6/10. All five task groups were complete before analysis. The records identify specific scenarios and stopping reasons for further diagnosis, but do not show that articulation alone causes the failures.

The contrast between these tasks and simpler placement scenarios suggests that a single aggregate score conceals important differences in grounding, accessibility, and stage composition. It also cautions against interpreting the suite name as a reliable ordering of control difficulty. The task-wise counts expose these differences without changing the planned sample weights.

B. Controller Completion Is a Different Outcome

Table III crosses terminal controller state with external success. In 11 episodes the controller reports completion while the evaluator reports failure. Conversely, 35 episodes receive external success without controller completion. The combined disagreement is 46/400 (11.5%). The latter group includes cases where a predicate is reached before a later local failure or budget exhaustion. Because the recorder retains an ever-success latch, these records do not imply successful final-state verification.

This difference has a practical implication for reporting and controller design. Using the controller’s local success flag as the benchmark label would produce another measurement. Using external success to advance the controller would change its information interface. Our analysis keeps both values and makes neither substitution. The aggregate records do not retain the first time of external success or a separate final-only label, so they cannot quantify how much continued execution changes final-state success or how much runtime an early-stop evaluator would save.

TABLE IV

OUTCOME GROUPS AMONG EXTERNAL FAILURES. CATEGORIES ARE DERIVED FROM RECORDED CONTROLLER MESSAGES WITH A FIXED PRECEDENCE RULE.

Recorded outcome group	Episodes
Phase timeout	17
Controller completion / external failure	11
Sensor verification rejection	9
Global action budget	7
Other controller failure	6
Grasp / retention rejection	3

C. Recorded Failure Categories

Table IV groups the 53 externally unsuccessful episodes using their persisted status and failure message. The grouping gives controller completion/external failure and global-budget exhaustion precedence; remaining messages are classified by phase timeout, grasp/retention rejection, sensor-verification rejection, or another failure. This is a reproducible description of the recorded stopping reason, not a manually verified taxonomy of physical root causes.

A phase timeout may arise from an inaccessible target, calibration error, occlusion, or contact resistance. Likewise, a rejected sensor verification may reflect an actual placement error or perception uncertainty. Determining these causes requires frame-level or geometric-trace analysis. The current evidence supports identifying where execution terminates, while leaving causal attribution open.

D. Execution Cost and Qualitative Context

The median newly executed episode takes 66.5 seconds. Mean action counts range across suites as shown in Table II. Long episodes average 293.3 actions, compared with 173.0 for Object. These costs include controller execution through its own stopping condition; they are not time-to-first-success. Recorded elapsed times mix perception, simulation, rendering, and video overhead while workers compete for the same CPU and GPU. They should not be interpreted as real-robot control frequency.

The run reports 0 runtime exceptions. Dispatcher wall time is 70.2 minutes for the campaign containing 390 new and ten reused episodes. We have no matched serial run with the same complete task mix and therefore make no measured parallel-speedup claim.

Figure 3 provides visual context from actual recordings. The upper row shows a successful Object task-00 episode. The lower row is the earliest externally unsuccessful Goal task-03 episode in the official index order, which exhausts its action budget. Selection is stated to make the figure interpretable as illustration, rather than a representative sampling estimate or evidence of a particular failure mechanism.

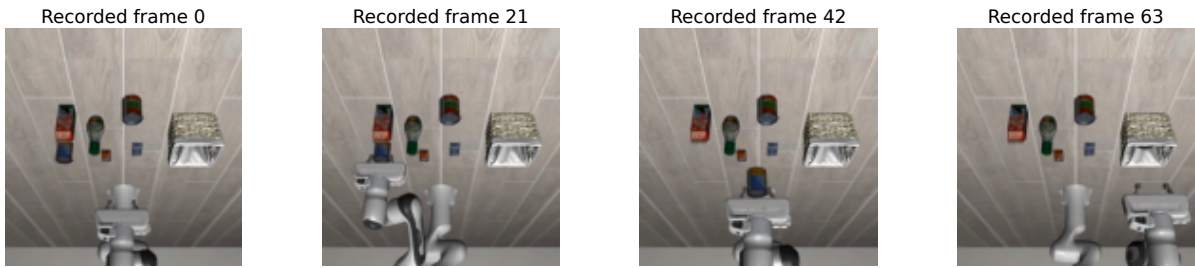
VI. DISCUSSION AND LIMITATIONS

The results show that explicit geometry and specialized skills can solve a substantial fraction of the selected manipulation episodes under declared asset and calibration assumptions. Their value as a baseline lies partly in the

Spatial	10/10	8/10	10/10	9/10	5/10	9/10	10/10	10/10	10/10	10/10
Object	10/10	9/10	10/10	10/10	9/10	10/10	10/10	8/10	10/10	10/10
Goal	7/10	10/10	10/10	3/10	10/10	10/10	8/10	10/10	10/10	10/10
Long	8/10	9/10	9/10	6/10	8/10	9/10	9/10	10/10	4/10	0/10
	00	01	02	03	04	05	06	07	08	09
Official task index (within each suite)										

Fig. 2. External successes divided by recorded trials for each official task index. Columns identify different tasks in each suite; darker cells indicate higher success. Counts, rather than rounded percentages alone, show the sample size supporting each result.

Object 00 / init 00: external success



Goal 03 / init 02: action-budget exhaustion



Fig. 3. Actual fixed-camera frames from two recorded episodes. Top: *pick up the alphabet soup and place it in the basket* (Object 00, initial state 00). Bottom: *open the top drawer and put the bowl inside* (Goal 03, initial state 02). Four uniformly spaced recorder indices are shown; recording uses a stride of two actions. Images are crops of the fixed-camera half of the original dual-view video, with no synthetic image content.

visibility of those assumptions. Texture and size priors encode information about the known objects; grammar rules encode known language forms; and the controller’s many special cases reflect manual development. These resources must be accounted for when comparing the system with a learned policy trained on a different set of observations and demonstrations.

a) Generalization and novelty.: The run does not test unseen object families, held-out language constructions, new embodiments, depth noise, calibration perturbations, or physical hardware. It also does not provide a held-out history of controller development. Accordingly, the success rates characterize the deployed benchmark system, not an estimate of zero-shot task generalization. The individual geometric operations and state-machine mechanisms are established

techniques. The claimed contribution is their documented integration and the resulting measurement, not a new learning algorithm or a formal control guarantee.

b) Missing controlled comparisons.: There are no matched learned-policy baselines or component ablations in this run. In particular, the evidence does not estimate gains from asset priors, the second camera, carried-offset correction, or recovery logic. A follow-up study should hold initial states and evaluation semantics fixed while removing these components, and should report both internal completion and external success. Repeated environment seeds and deliberately perturbed assets would distinguish repeatability from transfer.

c) Reliability and interpretation.: Bounded actions and local guards limit the implemented commands, but do not

establish collision safety, contact-force safety, or real-world reliability. The narrow ten-trial task samples can also conceal rare failure modes. The observed controller/evaluator disagreement points to a specific measurement improvement: record first-success time and final predicate separately, then evaluate alternative stopping conventions without silently changing the policy’s observations.

d) Reproducibility.: Source and resource checksums, fixed initial-state identifiers, and record-derived figures support reconstruction of this evaluation. The software-boundary counters are implementation-generated records, rather than independent security attestations. The current artifact should therefore be used to reproduce and inspect a concrete system, with the experimental scope stated above.

VII. CONCLUSION

We presented Route B, an asset-informed geometric manipulation baseline that compiles supported language into closed-loop grasp–place and contact skills. Under its declared sensing and prior assumptions, the evaluated system achieves 86.8% external success across 400 recorded episodes. Task-level outcomes and completion disagreements reveal limitations that an aggregate success rate alone would hide. The implementation and record-based analysis provide a concrete starting point for controlled studies of geometric priors, contact recovery, and policy-evaluation interfaces.

ACKNOWLEDGMENTS AND AI DISCLOSURE

OpenAI Codex assisted in drafting and editing the text in all sections, preparing the LaTeX source, and writing scripts for the numerical tables and figures. Reported outcomes are computed from existing simulator evaluation records. The rollout images are extracted from those recordings; no generative image model was used.

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [2] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [3] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” *arXiv preprint arXiv:2306.03310*, 2023.
- [4] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated task and motion planning,” *arXiv preprint arXiv:2010.01083*, 2020.
- [5] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [6] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” *arXiv preprint arXiv:2209.07753*, 2022.
- [7] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3d value maps for robotic manipulation with language models,” *arXiv preprint arXiv:2307.05973*, 2023.
- [8] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, “ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation,” *arXiv preprint arXiv:2409.01652*, 2024.

- [9] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *European Conference on Computer Vision*, 2024.
- [10] OpenVLA Contributors, “LIBERO evaluation script and environment utilities,” <https://github.com/openvla/openvla>, accessed September 2026. Paths: `experiments/robot/libero/run_libero_eval.py` and `libero_utils.py`.
- [11] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.
- [12] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu, “robosuite: A modular simulation framework and benchmark for robot learning,” *arXiv preprint arXiv:2009.12293v3*, 2025.